

AICP: A Minimal Protocol for LLM-Native Agent Systems

A Unified Reduction of Actor, Message Bus, Event Sourcing, and Microkernel Paradigms

Dvwoo

GitHub: <https://github.com/woozheng/aicp>

October 6, 2026

Abstract

Contemporary agent frameworks accumulate complexity: state machines, multi-agent schedulers, tool registries, context buses, and lifecycle managers. Each solves a real problem, yet their combination imposes heavy cognitive load on both developers and large language models (LLMs).

We present AICP (Agent Interaction & Communication Protocol), a minimal protocol that reduces the canonical paradigms of distributed computing — Actor model, message bus, event sourcing, microkernel, plugin architecture, and dependency injection — into a single uniform structure: **messages flow, plugins react, state accumulates**.

AICP is defined by roughly 200 lines of core specification. It has no agent instances, no context bus, no registry, and no scheduler. Plugins share a single signature (`async def execute(envelop, agent)`), communicate exclusively through a single message type (`Envelop`), and require state to be append-only, with no separate mutable store.

We argue that this reduction is not merely aesthetic. It lowers the cognitive burden on LLMs by requiring them to internalize **one concept** — **message passing** — which covers three apparently distinct operations: **calling** an existing plugin, **creating** a new plugin, and **calling again** the newly created plugin. These are not three mechanisms; they are the same mechanism, applied at different moments. The mechanism is **self-similar in form**: a plugin calling another plugin, an LLM generating a plugin, and an external caller triggering a plugin all go through the same shape. The *effects* differ, but the differing effect is carried in the returned `Envelop`, so the LLM never needs to learn a second shape.

AICP makes a specific exchange. It gives up **provability** — it does not prove that negotiation produces a valid agreement, that plugins report honestly, or that computations terminate — and gains **cross-language identity** and **cross-node transparency**: a plugin written in one language is understood in another, and a plugin registered on one node is callable from another, neither requiring an adaptation layer. This exchange is not arbitrary: provability requires closure, and cross-language and cross-node reach require openness. A closed system can prove its properties; an open one can reach across boundaries. AICP chooses openness.

We introduce a **three-tier control plane** that separates protocol-level control from negotiable control from implementation-level control, and a **failure semantics** in which

termination is enforced by the framework, not entrusted to handlers. We state the protocol’s **boundaries** explicitly: it guarantees termination, not success; it guarantees convergence for messages, not for computations; it stays silent about routing policy for failed messages; it enables interoperability without guaranteeing it; and it assumes honest **status** reporting. We emphasize that AICP is entirely defined by its core — the **Envelop** type, the **route** function, the plugin signature, the plugin lookup space, and the append-only state constraint. Everything else is implementation.

1 Introduction

1.1 The Accumulation Problem

Modern agent frameworks tend to grow by addition. Each new requirement — tool calling, multi-agent coordination, memory, streaming, async callbacks — is met with a new abstraction: a scheduler, a context bus, a memory store, a tool registry, a lifecycle manager. Over time, the framework becomes a layered stack of mechanisms, each locally justified, collectively opaque.

This accumulation has three costs:

- **Cognitive cost for developers.** Understanding the system requires understanding every layer and its interactions.
- **Cognitive cost for LLMs.** Generating correct code requires holding many conventions simultaneously — signatures, return shapes, registration rules, lifecycle expectations.
- **Portability cost.** Each abstraction tends to be language- and runtime-specific, making cross-language replication a rewrite rather than a translation.

1.2 The Reduction Hypothesis

We hypothesize that most of these abstractions are not independent primitives but variants of a single underlying structure: a message-oriented system in which independent units react to messages, and state accumulates as an append-only record of those reactions.

If this is true, then a protocol built on this single structure — without the additional abstractions — should be able to express everything the layered frameworks express, while imposing far less cognitive load.

AICP is an attempt to test this hypothesis.

1.3 The Scope of the Protocol

AICP is defined by its **core**:

1. The **Envelop** type, with fields **sender**, **receiver**, **ttl**, **status**, **meta**.
2. The **route** function.
3. The plugin signature `async def execute(envelop, agent)`.
4. The plugin lookup space: plugins are found by **receiver** in a shared namespace.

5. The append-only state constraint.

Nothing else is part of the protocol.

In particular:

- Any particular LLM scheduler — however it prompts, parses, or validates LLM output — is an **implementation**.
- Any particular set of generation conventions — how large text is delimited, how reasoning is bounded, how sandboxes are enforced — is an **implementation**.
- Any particular tool for sub-agent management or contract discovery — `task_manager`, `contract_agent`, or their equivalents — is an **implementation**.
- Any particular state store — an append-only list, a log, a table, a snapshot — is an **implementation**.
- Any particular realization of the plugin lookup space — a dictionary, a filesystem, a database, a name service — is an **implementation**.
- Any particular mechanism for forcing a return from a stuck plugin — thread interruption, process isolation, a watchdog — is an **implementation**.

We make this distinction explicit because it is the source of AICP’s minimality. The protocol is not “a framework with a small core.” It is a core, plus an open space in which implementations may differ.

1.4 Contributions

We reduce six canonical paradigms to a single protocol of roughly 200 lines.

We show that this protocol requires the LLM to learn **one concept** — **message passing** — which covers calling, creating, and re-calling alike, because the mechanism is **self-similar in form** across layers, with differing effects carried in the returned Envelop.

We introduce a **failure semantics** in which termination is enforced by the framework, not entrusted to handlers, together with a **three-tier control plane** whose boundary is drawn by a two-stage criterion that avoids circularity.

We state the protocol’s **boundaries explicitly**: termination not success; convergence for messages not computations; silence about routing policy for failed messages; interoperability enabled but not guaranteed; and honesty of **status** reporting assumed.

We state the protocol’s **exchange explicitly**: AICP gives up provability and gains cross-language identity and cross-node transparency. This exchange is not arbitrary; it is forced by a logical opposition between closure and reach.

We demonstrate one-shot LLM code generation, cross-language structural identity, self-bootstrapping, and cross-domain generation from a single protocol description.

2 Related Work

2.1 The Actor Model

The Actor model (Hewitt, 1973; Agha, 1986) introduced the idea of independent computational entities communicating solely via messages, with no shared state. Erlang (Armstrong, 1986) demonstrated its practicality at scale.

AICP preserves the message-passing core of Actor systems but removes the Actor as an entity. There is no actor object, no mailbox, no PID. A “plugin” is a function; a “session” is an identifier; state is append-only.

Unlike the Actor model, AICP does not enforce actor identity or mailbox semantics. This is a deliberate trade: AICP gains determinism and observability at the cost of actor autonomy. Address passing is restricted — addresses are carried in `meta` by convention, not as first-class protocol concepts. As we discuss in §3.8–§3.10, this restriction is bounded by a failure semantics that guarantees termination, and by a three-tier control plane that makes coordination expressible without making it mandatory.

2.2 Message Buses and RPC

Enterprise message buses and RPC systems (CORBA, gRPC) provide a central dispatch layer with registered endpoints. AICP retains dispatch but eliminates the bus as an object: the dispatcher is a lookup in the plugin lookup space followed by a function call.

2.3 Event Sourcing

Event sourcing (Fowler, 2005) treats state as the fold of an event log. AICP adopts the append-only constraint directly, but does not prescribe a particular realization. Unlike typical event sourcing, AICP does not separate “events” from “state projections” — the append-only record is read directly by the LLM, with per-entry retention hints (an implementation choice) controlling what is surfaced.

2.4 Microkernels and Plugin Architectures

Microkernels (Liedtke, 1995) minimize the kernel and push functionality into user-space servers. Plugin architectures (e.g., Eclipse, VS Code) register extensions against a host. AICP combines both: the “kernel” is a routing function; “plugins” are found in a shared namespace. There is no registry — the lookup space is the registry, and its realization is an implementation choice.

2.5 Dependency Injection

DI frameworks inject capabilities into components. AICP requires that capabilities are injected, not imported: plugins receive an `agent` object rather than importing capabilities themselves. The specific capability set is an implementation choice.

2.6 MCP and Tool Protocols

The Model Context Protocol (MCP) standardizes how LLMs call predefined tools. AICP differs fundamentally: tools (plugins) are not predefined — they are generated, inspected, and repaired at runtime by the LLM itself. AICP is not a tool-calling protocol; it is a tool-creating protocol. And because creating a tool is itself a call in form, the distinction between calling and creating dissolves at the level of form.

2.7 CyberOrgs and Resource-Bounded Agents

CyberOrgs (Jamali, 2004) models resource-bounded multi-agent computation over peer-owned networks. It treats control as a first-class abstraction: each cyberorg owns resources and eCash, and hosts other cyberorgs under negotiated contracts. The model provides an operational semantics and proves termination properties.

AICP shares with CyberOrgs the concern for analyzability, but differs in scope. CyberOrgs reads as a complete system design: it reifies control, resources, currency, and negotiation into a single coherent model, from operational semantics down to a prototype implementation. That completeness is what makes its transition system and its termination properties possible. It is also what makes it a single system rather than a shared vocabulary: how two independently deployed CyberOrgs would trade resources across their boundaries is not a question the model answers, because the model does not have to — it owns the whole system.

AICP starts from a different place. It is not a system but a protocol for agents that were never deployed together and may not share an implementation. The question AICP had to answer was not “what should control be” but “what must control be reified *into*, if the protocol is to remain a protocol and not become a system”. The answer is Tier 1 only — `ttl` and `status`. Everything else is Tier 2, which the protocol makes *expressible* but not *enforced*. Where CyberOrgs proves dormancy through resource exhaustion, AICP proves termination through `ttl` decrement. Where CyberOrgs enforces coordination through contracts, AICP makes coordination expressible through `meta` exchange. The difference is one of starting point, not of quality.

2.8 Process Calculi

Process calculi (Milner, 1999) formalize concurrent systems as processes exchanging names over channels. AICP shares with process calculi the idea that communication is the primitive, and that structure emerges from communication patterns rather than being imposed by the language.

AICP differs in that it is not a calculus but a protocol. It does not attempt to be a complete formal system; it defines a minimal set of message-shape and control-layer constraints that a runtime must enforce. A formal semantics for AICP — in the style of a labeled transition system over Envelop states — is a natural direction for future work, and would establish the termination claims of §3.8 as theorems rather than design arguments (see §10.8).

3 The AICP Protocol

3.1 The Core

AICP is defined by five things, and only five things:

1. **The Envelop type** — the sole message type, carrying `sender`, `receiver`, `ttl`, `status`, `meta`.
2. **The route function** — the sole routing function.
3. **The plugin signature** — `async def execute(envelop, agent)`, where `envelop` is the message and `agent` is the injected capability container.
4. **The plugin lookup space** — plugins are found by `receiver` in a shared namespace.
5. **The append-only state constraint** — state is append-only; there is no separate mutable state store.

Everything else in this paper is either (a) a consequence of these five, or (b) an implementation choice that is not part of the protocol.

3.2 Envelop

`Envelop` is the sole message type. It carries:

- `sender`, `receiver` — routing identity
- `payload` — business content
- `ttl` — lifecycle
- `status` — protocol-defined failure state (§3.8)
- `meta` — negotiable control plane (§3.10)

`status` **handles errors only**. It does not carry business state. Its semantics are:

- **Empty status** — no protocol-level error occurred.
- **Non-empty status** — the value must be one of the protocol-defined failure classes (§3.8).

Business state travels in `payload`, never in `status`. This distinction keeps the protocol’s failure semantics finite and analyzable: because `status` has finitely many values, the protocol can define a complete transition system over them.

meta is the negotiable control plane. It is not business content. It is the plane on which conventions exchange control — callbacks, sessions, signatures, negotiation proposals, namespace membership. It is not a second message type; all communication is still a single `Envelop`.

There is no intent field. An implementation that needs to dispatch within a plugin may carry a dispatch label in `meta`; the protocol does not name such a label, because naming it would introduce a second routing dimension and violate the one-shape principle.

No other field is part of the protocol. Implementations may add fields, but the protocol does not see them.

3.3 Route

Routing is a single function:

```
1 async def route(envelop, agent):
2     plugin = plugins.get(envelop.receiver)
3     return await plugin(envelop, agent)
```

The protocol requires that `route`:

1. Look up the plugin by `envelop.receiver` in the plugin lookup space.
2. Invoke it with `(envelop, agent)`.
3. Return the result.
4. **Consume one ttl for every routing attempt** (§3.8).

The **realization** of the lookup space is an implementation choice. The **requirement** that plugins are found by `receiver` in a shared namespace is protocol-level: without it, `MISSING` failures would be unanalyzable, and two independently generated plugins could not call each other.

Routing modes — synchronous, asynchronous with callback, callback acknowledgement — are distinguished by `meta` (§3.10). They are not protocol primitives; they are conventions expressed in Tier 2.

3.4 Plugins

A plugin is a Python (or TypeScript) function:

```
1 async def execute(envelop, agent):
2     # status remains "" -- no protocol-level error
3     envelop.payload = {"ok": True, "data": {...}}
4     return envelop
```

There is no class, no instance, no lifecycle. A plugin “exists” iff it is found in the plugin lookup space.

A plugin may return an `Envelop` with `status` empty (success), `EXCEPTION` (it raised), or `META_FAILED` (it refuses or signals a meta disagreement). It may not return `MISSING`, `TIMEOUT`, or `INVALID` — those are produced by `route`, not by plugins.

3.5 State

The protocol requires that state is **append-only** and that there is **no separate mutable state store**. This follows from the absence of shared mutable state: plugins cannot communicate through side effects on a shared store; they communicate only through messages.

How this is realized is an implementation choice. The reference implementation uses an **InformationFlow** — an append-only list of entries, persisted as JSON, with per-entry retention hints. Another implementation may use a log, a table, a snapshot, or any other append-only structure.

The protocol sees only the constraint, not the realization.

3.6 Capability Injection

The protocol requires that capabilities are **injected**, not imported. Plugins receive an **agent** object rather than importing capabilities themselves.

What capabilities the **agent** object exposes is an implementation choice.

3.7 The Absence of a Context Bus

Traditional agent systems maintain a context bus: a central object holding registered agents, session state, and global context.

AICP has no such object. “Context” is the append-only record. “Sessions” are identifiers. “Agents” are functions. The bus is replaced by two primitives: **route** and the plugin lookup space.

We argue this absence is not a simplification but a dissolution: the concept of a context bus is a consequence of treating agents as instances. Once agents are functions, the bus has nothing to hold.

3.8 Failure Semantics

AICP distinguishes **structural failure** from **semantic failure**.

3.8.1 Structural Failure

Structural failure is produced by the protocol. It has exactly four classes:

Class	Produced by	Trigger	Transition
MISSING	route	receiver has no plugin	→ PENDING (retry; consumes ttl)
TIMEOUT	route	a routing attempt fails to produce a result within its time bound	→ RETRY (retry; consumes ttl)
EXCEPTION	plugin	plugin raises	→ ISOLATED (no further attempts)
INVALID	route	Envelop violates constraints	→ DROPPED (no further attempts)

These classes are **protocol-level** because the protocol itself produces them.

Each routing attempt consumes one ttl. A routing attempt occurs whenever **route** is invoked, whether the invocation succeeds, fails, or enters a waiting state. The protocol does not prescribe the interval between attempts; it prescribes only that each attempt consumes one **ttl**. **The unit of ttl is an implementation choice:** two implementations may exhaust

`ttl` in different wall-clock times. The protocol guarantees eventual termination, not a specific duration.

DROPPED, ISOLATED, and DORMANT are **terminal**. PENDING and RETRY are **non-terminal** and are driven by **framework-scheduled retries**. Each retry is a routing attempt and consumes one `ttl`. The scheduling of retries is an implementation choice; what is protocol-level is that retries occur, and that each consumes `ttl`.

Therefore, even a message in PENDING consumes `ttl` on every retry, and when `ttl` reaches zero it enters DORMANT. No message can wait indefinitely.

3.8.2 Semantic Failure

Semantic failure is produced by conventions — specifically, by disagreement over the contents of **meta**. The protocol does not know what **meta** contains, so it cannot classify these failures directly. Instead, the protocol defines a single state:

- **META_FAILED** — a handler has signaled that the **meta** contract could not be satisfied, or has refused the Envelop.

When a message enters **META_FAILED**, the protocol guarantees two things:

1. The message's `ttl` continues to decrement on every routing attempt.
2. **The framework, not the handler, enforces termination.** If the handler does not move the message to RECOVERED or ISOLATED before `ttl` reaches zero, **route** forces the message to DORMANT.

The handler may choose the **content** of recovery. But the handler cannot prevent the `ttl` from decrementing, and cannot prevent the framework from enforcing the terminal transition when `ttl` is exhausted.

The framework guarantees termination, not success. If negotiation succeeds before `ttl` is exhausted, the message reaches RECOVERED. If it does not, the message reaches DORMANT. Both are defined outcomes. The protocol does not promise that **META_FAILED** leads to agreement; it promises that it leads to a terminal state.

RECOVERED is a non-terminal state indicating that the **meta** disagreement has been resolved. When it is reached, **status** is cleared to empty, and the message continues on the normal path.

Refusal. A handler may refuse an Envelop by returning **status** = **META_FAILED**. Refusal is not an error; it is a legitimate protocol action. It signals that the handler does not accept the current **meta** convention.

The protocol defines the states of failure. The implementation defines the content of recovery. The framework enforces termination regardless of the content.

3.9 The Boundaries of Failure

Three boundaries complete the failure semantics.

3.9.1 Failed Messages Stay with Their Caller

The protocol does not move a failed message to a different receiver. `route` is a synchronous function: when it sets a `status`, it returns the Envelop to its caller. The caller holds it, and decides what to do next — retry, generate a handler, or let it reach a terminal state.

This is deliberate. A protocol that moved failed messages elsewhere would need to know where — and that knowledge would be a convention the protocol cannot own. By keeping the failed message with the caller, the protocol stays silent about routing policy while still guaranteeing that the message is *somewhere* defined.

`sender` and `receiver` are not swapped on failure. They retain the values they had when `route` was invoked. The only exception is the asynchronous callback case, which is a Tier 2 convention: when a handler completes asynchronously, it constructs a callback Envelop whose `sender` is the original receiver and whose `receiver` is the callback address. That construction is the implementation's choice, not a protocol rule.

In short: **the protocol guarantees that a failed message is returned to its caller, not that it is rerouted.** Rerouting is a Tier 2 convention expressed in `meta`.

3.9.2 Convergence for Messages, Not Computations

The protocol assumes that a plugin eventually returns. A plugin that never returns — an infinite loop, a blocked call, a hung process — is not something the protocol itself can handle, because `route` never regains control and `ttl` cannot be decremented.

This is the boundary of the protocol's layer. The protocol is a **message layer**. It guarantees termination for every message it can see. A plugin that does not return is, at the message layer, invisible.

The framework must **have the ability** to force a return from a stuck plugin — this is Tier 1, because without it `ttl` cannot decrement and termination fails. **How** the return is forced — thread interruption, process isolation, a watchdog — is Tier 3.

When the forced return occurs, `route` regains control and sets `status = TIMEOUT`. From that point, the protocol's failure semantics apply normally.

AICP guarantees termination for messages, not for computations. The protocol's guarantee is conditional on the framework's ability to force a return.

3.9.3 Honest Status Reporting Is Assumed

A plugin may lie. It may return `status = ""` when it has actually failed, or `status = "EXCEPTION"` when it has succeeded. The protocol cannot detect this, because it cannot verify a plugin's internal state.

The protocol assumes that plugins report their `status` honestly. A plugin that lies is outside the protocol's guarantee. This is not a defect of the protocol; it is a consequence of the protocol being a message layer: it sees messages, not intentions.

3.10 The Three-Tier Control Plane

AICP distinguishes three tiers of control.

Tier 1 — Protocol-level control. Controls that the protocol produces, that must be analyzable on failure, and whose specific form the protocol must know in order to define transitions. These are Envelop fields: `ttl`, `status`.

Tier 2 — Negotiable control. Controls that must be shared across generators to coordinate, but whose specific form the protocol need not know. These live in `meta`. Examples: `trace_id`, `message_id`, `callback_receiver`, `session_id`, `signature`, negotiation proposals, namespace membership protocols, and any dispatch label an implementation chooses.

Tier 3 — Implementation control. Controls that are neither produced by the protocol nor needed across generators. These are the implementation’s own.

The boundary between tiers is determined in **two stages**. The two-stage structure avoids circularity: the criteria operate at different stages and do not presuppose each other.

Stage 1 — Filter. A control is a *candidate for protocol-level treatment* if both of the following hold:

1. **Analyzable on failure.** If the control is undefined, failure becomes unanalyzable.
2. **Shared across generators.** It must be shared across independently generated handlers.

Controls that fail Stage 1 are Tier 3.

Stage 2 — Classify. Among the candidates, a control is Tier 1 if the protocol must know its **specific form** in order to define transitions. It is Tier 2 if the protocol needs only to know that the control **exists as an exchangeable value**.

We note that “analyzable on failure” is a **design criterion, not a mechanical test**. In cases of doubt, the protocol errs toward Tier 2, because Tier 2 is where coordination conventions live, and coordination is what the protocol makes expressible rather than mandatory.

Applying the stages:

- `ttl`, `status` — **Tier 1**.
- `trace_id`, `message_id`, `callback_receiver`, `session_id`, `signature`, negotiation proposals, namespace membership protocols — **Tier 2**.
- Implementation conventions, sandbox rules, state stores, lookup-space realizations, execution timeouts, retry schedules — **Tier 3**.

Control that the protocol must know in form is reified as Envelop fields. Control that must be shared but need not be known in form is reified as `meta` exchange. Control that need not be shared at all is not reified.

3.11 What Is Not Part of the Protocol

The following are implementation choices, not AICP:

- How an LLM scheduler prompts, parses, or validates LLM output.
- How large text is delimited (e.g., `@@CONTENT@@`).
- How reasoning length is bounded (e.g., `think` length).
- How a sandbox is enforced (e.g., replacing builtins).
- How sub-agents are managed (e.g., `task_manager`).
- How plugin contracts are discovered (e.g., `contract_agent`).
- What capabilities the `agent` object exposes.
- How state is stored (e.g., `_flow`, a log, a table, a snapshot).
- How the plugin lookup space is realized (e.g., a dictionary, a filesystem, a database, a name service).
- How the shared namespace is maintained across processes or hosts.
- How a forced return from a stuck plugin is implemented.
- How retries are scheduled.
- The unit of `ttl`.
- What additional fields an `Envelop` carries.
- Any dispatch label within a plugin (e.g., `intent`).

These are all **Tier 3**.

4 Protocol Enforcement

4.1 What the Protocol Must Enforce

A protocol is only a protocol if it is enforced. AICP requires an implementation to enforce exactly five rules:

- **Envelop structure.** Every `Envelop` must carry `sender`, `receiver`, `ttl`, `status`, `meta`.
- **Plugin shape.** Only `async def execute(envelop, agent)` is accepted as a plugin.
- **Plugin lookup.** Every plugin is reachable by `receiver` in a shared namespace.
- **Append-only state.** State must be append-only; no separate mutable state store may exist.
- **Failure termination.** Every plugin must return an `Envelop` whose `status` is empty, `EXCEPTION`, or `META_FAILED`. Any other `status` triggers `INVALID`. The framework must

enforce terminal transitions when `ttl` is exhausted. The framework must **have the ability** to force a return from a stuck plugin, producing `TIMEOUT`.

These are the only rules the protocol itself imposes. They are Tier 1.

4.2 Enforcement, Not Documentation

Each protocol rule is enforced mechanically:

Rule	Enforcement
Envelop structure	Runtime check on every message
Plugin shape	Signature check before registration
Plugin lookup	Every registered plugin is indexed by receiver
Append-only state	No mutable store exposed to plugins
Failure termination	Runtime check on returned Envelop; unrecognized status $\rightarrow$ <code>INVALID</code> ; framework-enforced terminal transition on <code>ttl</code> exhaustion; framework able to force return from a stuck plugin

4.3 What the Protocol Does Not Enforce

An implementation may enforce additional rules — sandbox restrictions, output format requirements, generation constraints — but these are **not protocol rules**. A different AICP implementation may enforce entirely different rules and remain AICP-compatible, provided it enforces the five Tier 1 rules of §4.1.

This is what makes AICP minimal.

5 Cognitive Load Reduction: One Concept

5.1 One Concept

AICP requires the LLM to internalize **one concept**: message passing. An Envelop is sent to a receiver; a plugin processes it; an Envelop is returned. This action has no variants in form.

5.2 What Counts as an Operation

We define an **operation** as an external caller — the LLM, or a plugin — sending an Envelop to a receiver and receiving an Envelop in return.

Under this definition:

- A plugin sending an Envelop internally is an operation of that plugin. From outside the plugin, it is invisible; the plugin’s own operation is still the single Envelop it received and the single Envelop it returns.
- Appending to the state record is **not** an operation in this sense. It is an implementation of the append-only constraint, not a message-passing act.
- Mutating the **agent** object is **not** an operation. **agent** is a capability container, not a communication channel.

By defining “operation” this way, the claim in §5.3 becomes a claim about what callers do, not about everything a plugin might do internally. It is a claim about the protocol’s surface.

5.3 Self-Similarity of Form

Calling, creating, and re-calling are not three operations in form. They are the same operation, applied at different moments:

- **Calling.** Send an Envelop to an existing plugin.
- **Creating.** Send an Envelop to a plugin that generates plugins. Creating is itself a call.
- **Re-calling.** The newly created plugin receives an Envelop, and is indistinguishable in form from any other plugin.

All three go through the same shape: an Envelop through **route** into **execute**, receiving (**envelop**, **agent**), returning an Envelop.

We call this **form self-similarity**. The *form* is identical at every layer.

The **effects** are not identical. A normal call returns an Envelop; a call to a generator plugin also returns an Envelop, but as a side effect it registers a new plugin in the lookup space. The differing effect is carried in the returned Envelop: the generator’s Envelop carries the receiver of the newly created plugin. The caller reads it and continues with the same shape.

So the LLM does not need to learn that creation has a different shape. Creation has the same shape; only the returned content differs.

We distinguish this from **effect self-similarity**, which AICP does not claim. Effects are not identical; only forms are. The weaker claim is the one that matters for cognitive load, because the LLM learns the form, and the form is what it must reproduce.

The assertion that all operations reduce to **route** entering **execute** is a claim about the protocol’s surface, not a theorem about its semantics. It can be read as a hypothesis: if a fourth operation were proposed that did not fit this form, the protocol would need to be revised. We regard this as the correct posture for a protocol. A formal statement — an operational semantics plus a reduction lemma — is left as future work (§10.9).

5.4 Opacity of Implementation

The internal implementation of a plugin is opaque to the LLM. The LLM needs to know only:

- the plugin’s **receiver**,
- what Envelop to send,
- what Envelop to expect back.

What the plugin does internally is the plugin’s own affair. This is what “the concrete implementation lives in the plugin” means: the protocol governs how messages flow, not what plugins do inside.

5.5 Why This Is a Reduction

A traditional framework requires the LLM to learn two or more conventions: one for calling tools, one for creating tools, one for orchestrating them. AICP requires one.

The reduction is not merely quantitative. In a layered framework, the meta-level and the object-level are structurally distinct, so a rule learned at one does not transfer to the other. In AICP, the form is the same, so a rule transfers automatically.

5.6 A Clarification

“One concept” refers to the **form of the action**, not to the **totality of knowledge**. The LLM still discovers what each plugin does at runtime. It does not need to learn how to call. The concept is constant; the content is discovered.

5.7 Design Observations

The following are design observations, not formal measurements.

- **One-shot correctness.** In practice, plugins generated against AICP are accepted on first attempt more often than against layered frameworks.
- **Minimal context.** Generating a plugin requires the protocol description (a few hundred tokens), not framework documentation.
- **Cross-language replication.** Two independent implementations (Python and TypeScript) were produced by feeding the protocol and the reference implementation to an LLM; both are structurally identical to the original.

6 Self-Bootstrapping

6.1 Creation, Inspection, Repair

AICP supports a loop the LLM can run on its own plugin ecosystem: create $\rightarrow$ inspect $\rightarrow$ use $\rightarrow$ evaluate $\rightarrow$ repair. The loop is closed entirely within the agent; no human intervention is required.

Because creation has the same form as calling, this loop is not a meta-level operation in form. It is ordinary message passing, applied to plugins that operate on plugins. This is why self-bootstrapping does not require a hierarchy of meta-levels.

6.2 Self-Inspection

A plugin that inspects plugins can inspect itself. This is not a special case but a consequence of the absence of privileges: no plugin is special.

We take this self-inspection to be a minimal criterion for a self-bootstrapping agent system.

6.3 Sub-Agents Without Sub-Agent Machinery

Parallelism is achieved not by spawning agent instances but by invoking the main agent with a different session identifier. “Sub-agents” are thus a naming convention, not a mechanism.

How the required metadata is packaged is an implementation choice. It is Tier 3.

6.4 Failure Recovery Without Failure Machinery

Failure recovery follows the same pattern. The protocol defines failure states (§3.8); it does not define failure handlers. A handler for `MISSING` is a plugin that generates the missing plugin. A handler for `META_FAILED` is a plugin that negotiates the meta disagreement. Neither is special; both are generated when needed.

The framework guarantees termination; the implementation decides what the recovery contains.

7 The Meta-Model

7.1 Everything Is Message Flow

AICP reduces six paradigms to a single meta-model:

Paradigm	AICP Expression
Actor	Plugin reacting to Envelop
Message bus	<code>route(envelop, agent)</code>
Event sourcing	Append-only state (no separate mutable store)
Microkernel	Minimal core + plugin files
Plugin architecture	Plugins found by receiver in a shared namespace
Dependency injection	Capabilities injected, not imported

The meta-model is: **messages flow; plugins react; state accumulates.**

7.2 The Boundary of the Meta-Model

No paradigm-specific machinery is retained. What remains is the intersection of the paradigms — the smallest structure that expresses all of them.

The protocol’s core is exactly this intersection. Everything beyond it is implementation.

8 Cross-Domain Evidence

A single protocol description was provided to an LLM, which then generated systems in six unrelated domains:

No domain-specific training data and no framework documentation were provided. The protocol was the sole specification.

We do not claim these systems are production-grade. We claim the protocol is sufficient to convey the domain-independent structure an LLM needs to begin generating in a new domain.

Domain	System
Operating systems	Microkernel with processes, memory, FS, IPC, scheduler
Quantum computing	Simulator with qubits, gates, Shor code, VQE
Computational biology	Protein folding with multi-agent molecular dynamics
Machine learning	3D-parallel LLM trainer with All-Reduce and ZeRO
Number theory	Riemann Hypothesis exploration (Riemann–Siegel, Montgomery, GUE)
Hardware design	AI chip with ISA, compiler, chiplet interconnect

Note that in each domain, the generated system includes its own implementation conventions, its own state store, and its own lookup-space realization. These differ across domains. The protocol is what they share.

A concrete interoperability example. Consider two independently generated agents: A, which generates a compute plugin, and B, which generates a storage plugin. They have never negotiated.

Discovery. A must find B’s storage plugin by name. The protocol requires that both A and B have access to a **shared namespace** — a lookup space in which a plugin registered by B is visible to A by its **receiver**. The protocol does not prescribe how this namespace is realized. What it requires is that the namespace is **queryable by both**.

Invocation. A sends an Envelop whose receiver is B’s storage plugin name. **route** queries the shared namespace, finds the plugin, invokes it with (**envelop**, **agent**).

Disagreement. If A and B disagree on **meta**, the message enters **META_FAILED**. Each generates a negotiation plugin. The framework enforces termination: every routing attempt consumes **ttl**, and when **ttl** reaches zero the message enters **DORMANT** regardless of whether negotiation succeeded.

What this shows. Two agents that share the core can interoperate *provided* they also agree on how to join a common namespace. Sharing the core is necessary but not sufficient. The agreement is a **Tier 2** convention. Two agents that share the core but not a membership protocol will see each other’s plugins as **MISSING** — a defined, analyzable outcome, not a silent failure.

The **realization** of the namespace remains Tier 3.

9 Positioning

9.1 Relation to MCP

MCP standardizes tool invocation. AICP standardizes tool creation, inspection, and repair. The two are complementary. Because creating is itself a call in form, AICP does not need a separate “creation protocol.”

9.2 Relation to Traditional Agent Frameworks

Traditional frameworks accumulate mechanisms. AICP removes them. The claim is not that AICP does more; it is that AICP does the same with less.

9.3 Relation to Serverless and HTTP

AICP shares with serverless the absence of instance management and with HTTP the absence of session state at the protocol level. It extends both by treating the agent itself as a serverless, stateless unit.

9.4 Relation to CyberOrgs

CyberOrgs reifies control fully: contracts, resources, eCash, negotiation. AICP reifies control partially: only what the protocol must know in form, plus a negotiable plane for what conventions produce.

The difference is one of scope. CyberOrgs addresses resource-bounded computation over peer-owned networks; AICP addresses coordination among independently generated agents. CyberOrgs enforces coordination; AICP makes coordination expressible and enforces only termination.

9.5 Versioning and Compatibility

AICP’s core is stable: the Envelop structure, **route**, the plugin signature, the plugin lookup space, the append-only state constraint, the failure semantics, and the three-tier control plane.

Tier 2 (the contents of **meta**) and Tier 3 (implementation) may evolve freely. Two AICP implementations need only share the core to **speak the same protocol**.

To **interoperate**, two implementations must additionally agree on the Tier 2 conventions the interaction requires — most importantly, the membership protocol by which they join a common namespace. This agreement may be pre-arranged, negotiated at runtime, or established by a third party. The protocol does not prescribe which.

The precise sense in which “sharing the core” enables interoperability is this: it makes interoperability **expressible**. It gives two implementations a common vocabulary in which to state their Tier 2 agreements. It does not make interoperability **attainable** by itself — that requires the agreements to actually be reached, and reaching them is outside the protocol.

The distinction matters because it locates responsibility correctly: the protocol guarantees the vocabulary, not the conversation.

The core itself is versioned and may be revised when a defect is found. A revision of the core is a new minor version, and implementations declare the version they conform to.

10 Discussion

10.1 Why “Nothing” Looks Like Nothing

A system with no kernel object, no context bus, no registry, and no scheduler looks empty. This appearance is not a defect: it is the consequence of removing every abstraction whose necessity was assumed rather than demonstrated.

10.2 The Cost of Reduction

Reduction has costs. There is no node-level optimization, no cross-node state synchronization, no long-lived agent identity. We regard these as acceptable: they are features of systems that require node-level concerns, which AICP does not address.

10.3 The Role of LLMs

AICP’s minimalism is a precondition for LLM-nativeness. The protocol is simple enough that an LLM can internalize it from a few hundred tokens and generate correct code without framework documentation.

10.4 Why These Constraints Are Protocol-Level

A recurring question is why certain constraints are protocol-level while superficially similar ones are not. The answer is the same in each case: the protocol must enforce the **property** it needs, and must leave the **realization** of that property to the implementation.

- **Failure semantics.** Structural failure is produced by the protocol, not by a convention. A missing plugin, an expired `ttl`, a malformed `Envelop` — these are produced by the protocol itself. The protocol therefore knows their types and can define their transitions.
- **Control.** The two-stage criterion (§3.10) separates control the protocol must know in form from control it must only make exchangeable. Reifying all control would make AICP a framework.
- **State.** “Append-only” is a property the protocol needs in order to guarantee the absence of shared mutable state. The store — `_flow`, a log, a table — is a realization. A clarification: AICP has a shared append-only record; what it excludes is a shared *mutable* store.
- **Lookup space.** That two independently generated plugins can find each other by name is required for analyzability (`MISSING` failures depend on it) and for coordination. The realization — dictionary, filesystem, database — is not.

10.5 Protocol vs. Implementation

AICP’s central claim is that a protocol can be minimal and still analyzable, provided it distinguishes what it must know from what it must only make expressible.

This distinction is not always easy to maintain. The discipline is: apply Stage 1 (filter by failure-analyzability and cross-generator-sharing), then Stage 2 (classify by whether the protocol must know the specific form). Only controls that pass both stages are Tier 1.

10.6 The Boundaries of the Protocol

AICP's guarantees are precise, and so are its boundaries.

- **Termination, not success.** The framework guarantees that every message reaches a terminal state. It does not guarantee that `META_FAILED` leads to agreement; it guarantees that it leads to a terminal state.
- **Messages, not computations.** A plugin that never returns is outside the protocol's reach; it is handled by the framework's ability to force a return, which produces a Tier 1 `TIMEOUT`.
- **Failed messages stay with their caller.** The protocol guarantees a failed message is *somewhere* defined, not that it is rerouted.
- **Interoperability is enabled, not guaranteed.** Sharing the core makes interoperability expressible. Reaching it requires Tier 2 agreements the protocol does not provide.
- **Honesty is assumed.** A plugin that lies about its `status` is outside the protocol's guarantee.

These boundaries are not gaps. They are the consequences of the protocol being a message layer: it speaks about messages, and it is silent about everything that is not a message.

10.7 The Exchange

AICP makes a specific exchange, and it is worth stating plainly.

What AICP gives up: provability. AICP does not prove that negotiation produces a valid agreement, that plugins report their `status` honestly, or that computations terminate. These are properties a closed system can prove. AICP is not a closed system.

What AICP gains: cross-language identity and cross-node transparency. A plugin written in one language is understood in another. A plugin registered on one node is callable from another. Neither requires an adaptation layer, because the protocol does not distinguish languages or nodes — the core is five things, and those five things are the same shape in every realization.

Why the exchange is forced, not chosen. Provability requires **closure**: to prove a property of a system, you must own the system's boundary, its participants, and its resources. Cross-language and cross-node reach require **openness**: to reach across a language or a node, you must not assume a shared boundary. These two requirements are in logical opposition. A system cannot have both.

This is not a design preference; it is a structural fact. A closed system can prove its properties, but it cannot reach outside its boundary without an adaptation layer. An open system can

reach across boundaries, but it cannot prove properties it does not own. CyberOrgs chose closure and gained provability — its operational semantics and termination properties are the reward. AICP chooses openness and gains cross-language and cross-node reach — its structural identity across languages and its transparency across nodes are the reward.

Why, in the LLM era, the choice is not free. Provability has become unavailable, because plugins are generated at runtime by a generator that is not fully trusted. No protocol can prove the validity of an outcome when the participants are generated on the fly. Cross-language and cross-node reach have become necessary, because agents are generated independently, on different nodes, and in different languages. AICP does not choose openness because it is better. It chooses openness because it is the only remaining option — and it finds, in that option, two capabilities that closed systems cannot have.

What is preserved in the exchange. AICP does not give up all guarantee. It gives up **provability of outcomes** and retains **analyzability of process**. Every message reaches a terminal state. Every failure is classified and analyzable. Every coordination is expressible through a shared vocabulary. Every exchange leaves a defined trace, even when it fails. This is weaker than validity; it is stronger than silence.

The exchange is: from proving outcomes, to enabling reach.

10.8 Relation to Existing Formal Work

The termination claims of §3.8 are stated in the style of a protocol specification, not proved in the style of a semantics. They can be read as design arguments: the transition system is finite, each transition consumes `ttl`, and `ttl` is bounded, so every message reaches a terminal state. A formal proof would proceed by induction over routing attempts, and would need to make explicit the framework’s obligations (that retries occur, that `ttl` decrements on each attempt, that a stuck plugin can be forced to return). We regard this as future work, and we regard the current argument as sufficient for a design paper, insufficient for a formal-methods venue.

10.9 Open Problems

A formal operational semantics. A labeled transition system over Envelop states — in the style of Agha (1986) or Milner (1999) — would let us prove that every message reaches a terminal state, and characterize the framework’s obligations.

A reduction lemma for form self-similarity. §5.3 asserts that all operations reduce to `route` entering `execute`. This can be made precise: define an operational semantics, define what counts as an “operation” (we give a first definition in §5.2), and prove that every operation is an instance of the shape.

A granularity criterion for Stage 1. §3.10’s “analyzable on failure” is a design criterion, not a mechanical test. A precise characterization would make the Tier 1/2 boundary algorithmic.

Multi-agent discovery and membership. §8 and §9.5 locate namespace membership in Tier 2. What is not settled is whether a canonical membership protocol could be specified

without violating minimality, and whether two agents can negotiate membership through the same `META_FAILED` mechanism.

The validity of runtime-generated negotiation. When both parties to a negotiation are generated at runtime, by a generator that is not fully trusted, what does it mean for a negotiation to be *valid*? The protocol guarantees termination. It does not guarantee validity. This is the deepest open problem, and it is where the LLM era departs most sharply from prior work.

Adversarial plugins. §3.9 states two boundaries — plugins that do not return, and plugins that lie. A fuller treatment would characterize the class of adversarial behaviors a message layer can and cannot bound.

Empirical evaluation. §5.7 reports design observations, not measurements. A controlled study would convert them into evidence.

11 Conclusion

We have presented AICP, a protocol that reduces six canonical paradigms of distributed computing to a single meta-model: messages flow, plugins react, state accumulates.

The protocol is defined by its core: the `Envelop` type, the `route` function, the plugin signature, the plugin lookup space, and the append-only state constraint. It has no agent instances, no context bus, no registry, and no scheduler. It supports cross-language replication, one-shot LLM code generation, self-inspection, self-bootstrapping, and cross-domain generation.

We have argued that AICP lowers LLM cognitive load by requiring the LLM to internalize **one concept** — **message passing** — which covers calling, creating, and re-calling alike, because the mechanism is self-similar **in form** across layers, with differing effects carried in the returned `Envelop`.

We have introduced a failure semantics in which termination is enforced by the framework, not entrusted to handlers, and a three-tier control plane whose boundary is drawn by a two-stage criterion.

We have stated the protocol’s boundaries explicitly: termination not success; convergence for messages not computations; silence about routing policy for failed messages; interoperability enabled but not guaranteed; and honesty of `status` reporting assumed. These boundaries are as deliberate as the guarantees.

We have stated the protocol’s exchange explicitly: AICP gives up provability and gains cross-language identity and cross-node transparency. This exchange is forced by a logical opposition between closure and reach — a closed system can prove its properties but cannot cross boundaries without an adaptation layer; an open system can cross boundaries but cannot prove properties it does not own. In the LLM era, provability has become unavailable and cross-language, cross-node reach has become necessary. The exchange is not a preference; it is a consequence.

The protocol is minimal not because it ignores failure, control, or state, but because it distinguishes what it must know from what it must only make expressible.

The protocol is the soul; code is the body. The protocol defines what must be shared; code defines what may be invented.

A AICP Core (Reference Implementation)

Lines marked “Tier 3” are implementation choices, not protocol.

```
1 class Envelop:
2     def __init__(self, sender="", receiver="", payload=None, meta=None,
3         ttl=10):
4         self.sender = sender
5         self.receiver = receiver
6         self.payload = payload or {}
7         self.meta = meta or {}
8         self.ttl = ttl                # Tier 3: default value and unit
9         self.status = ""              # Tier 1: empty = no protocol-level
10        error
11
12 plugins = {}                         # Tier 3: lookup space realization
13
14 async def route(envelop, agent):
15     # Tier 1: every routing attempt consumes one ttl
16     if not envelop.receiver or envelop.ttl <= 0:
17         envelop.status = "INVALID"
18         return envelop
19     envelop.ttl -= 1
20
21     plugin = plugins.get(envelop.receiver)
22     if not plugin:
23         envelop.status = "MISSING"
24         return envelop                # framework may retry (each retry
25        consumes ttl)
26
27     try:
28         # Tier 3: forced return from a stuck plugin; Tier 1: the
29        ability to do so
30         result = await asyncio.wait_for(plugin(envelop, agent), timeout
31        =...)
32     except asyncio.TimeoutError:
33         envelop.status = "TIMEOUT"
34         return envelop
35     except Exception:
36         envelop.status = "EXCEPTION"
37         return envelop
```

```

34     # Tier 2: semantic failure signaled by convention
35     if result.meta.get("_meta_failed"): # Tier 3: key name
36         result.status = "META_FAILED"
37         # framework enforces termination: ttl keeps decrementing;
38         # if not resolved, framework forces DORMANT
39
40     return result
41
42 async def execute(envelop, agent):
43     # status remains "" -- no protocol-level error
44     envelop.payload = {"ok": True, "data": {...}}
45     return envelop

```

B Protocol Constitution (Tier 1)

- **Envelop structure.** sender, receiver, ttl, status, meta.
- **Plugin shape.** `async def execute(envelop, agent).`
- **Plugin lookup.** Plugins are found by receiver in a shared namespace.
- **Append-only state.** State is append-only; no separate mutable state store.
- **Failure termination.** Returned status must be empty, `EXCEPTION`, or `META_FAILED`. Any other status triggers `INVALID`. The framework enforces terminal transitions when ttl is exhausted, and has the ability to force a return from a stuck plugin (producing `TIMEOUT`).

C The Three-Tier Control Plane

Tier 1. Produced by the protocol; the protocol must know its form. Examples: `ttl`, `status`. Reified as Envelop fields.

Tier 2. Must be shared across generators; the protocol need not know its form. Examples: `trace_id`, `message_id`, `callback_receiver`, `session_id`, `signature`, negotiation proposals, namespace membership protocols. Reified as meta exchange.

Tier 3. Neither produced by the protocol nor needed across generators. Examples: LLM scheduler conventions, sandbox rules, sub-agent tools, contract discovery, state stores, lookup-space realizations, forced-return mechanisms, retry schedules, the `ttl` unit. Not reified.

D status Values

Non-terminal:

Terminal:

Value	Produced by	Meaning	Next
""	default	no protocol-level error	—
MISSING	route	receiver has no plugin	→ PENDING (retry, consumes ttl)
TIMEOUT	route	a routing attempt produced no result within its time bound	→ RETRY (retry, consumes ttl)
EXCEPTION	plugin	plugin raised	→ ISOLATED
INVALID	route	Envelop violated constraints	→ DROPPED
META_FAILED	plugin	meta contract unsatisfiable, or refusal	→ RECOVERED / ISOLATED / (framework forces) DORMANT
PENDING	route	awaiting plugin generation	retry (consumes ttl)
RETRY	route	awaiting retry	retry (consumes ttl)
RECOVERED	handler	meta disagreement resolved	→ ""

Value	Produced by	Meaning
DROPPED	route	message dropped
ISOLATED	route	message isolated
DORMANT	framework	ttl exhausted

status handles errors only. Business state travels in **payload**. Termination is enforced by the framework: every routing attempt consumes **ttl**, and when **ttl** reaches zero the framework forces DORMANT. The framework guarantees termination, not success.

E What Is Not Part of the Protocol

- LLM prompt format, output parsing, validation.
- Large-text delimiting (e.g., @@CONTENT@@).
- Reasoning length bounding (e.g., **think** length).
- Sandbox enforcement (e.g., replacing builtins).
- Sub-agent management (e.g., **task_manager**).
- Contract discovery (e.g., **contract_agent**).
- Capability set of the **agent** object.
- State storage (e.g., **_flow**, a log, a table, a snapshot).
- Lookup space realization (e.g., a dictionary, a filesystem, a database, a name service).
- Cross-process or cross-host namespace maintenance.
- Forced-return mechanism for stuck plugins.
- Retry scheduling.
- The unit of **ttl**.
- Additional Envelop fields.
- Any dispatch label within a plugin (e.g., **intent**).

Any AICP implementation may choose differently.

References

- [1] Agha, G. (1986). *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press.
- [2] Agha, G. (1990). Concurrent Object-Oriented Programming. *Communications of the ACM*, 33(9), 125–141.
- [3] Armstrong, J. (2007). *Programming Erlang*. Pragmatic Bookshelf.
- [4] Fowler, M. (2005). Event Sourcing.
- [5] Hewitt, C. (1973). A Universal Modular Actor Formalism for Artificial Intelligence. *IJCAI*.
- [6] Jamali, N. (2004). *CyberOrgs: A Model for Resource Bounded Complex Agents*. PhD thesis, University of Illinois at Urbana-Champaign.
- [7] Liedtke, J. (1995). On μ -Kernel Construction. *SOSP*.
- [8] Milner, R. (1999). *Communicating and Mobile Systems: the π -Calculus*. Cambridge University Press.
- [9] MCP Specification (Anthropic).
- [10] AICP Protocol Specification v5.9.
- [11] AICP Reference Implementations (Python, TypeScript).